Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Gaussian Processes

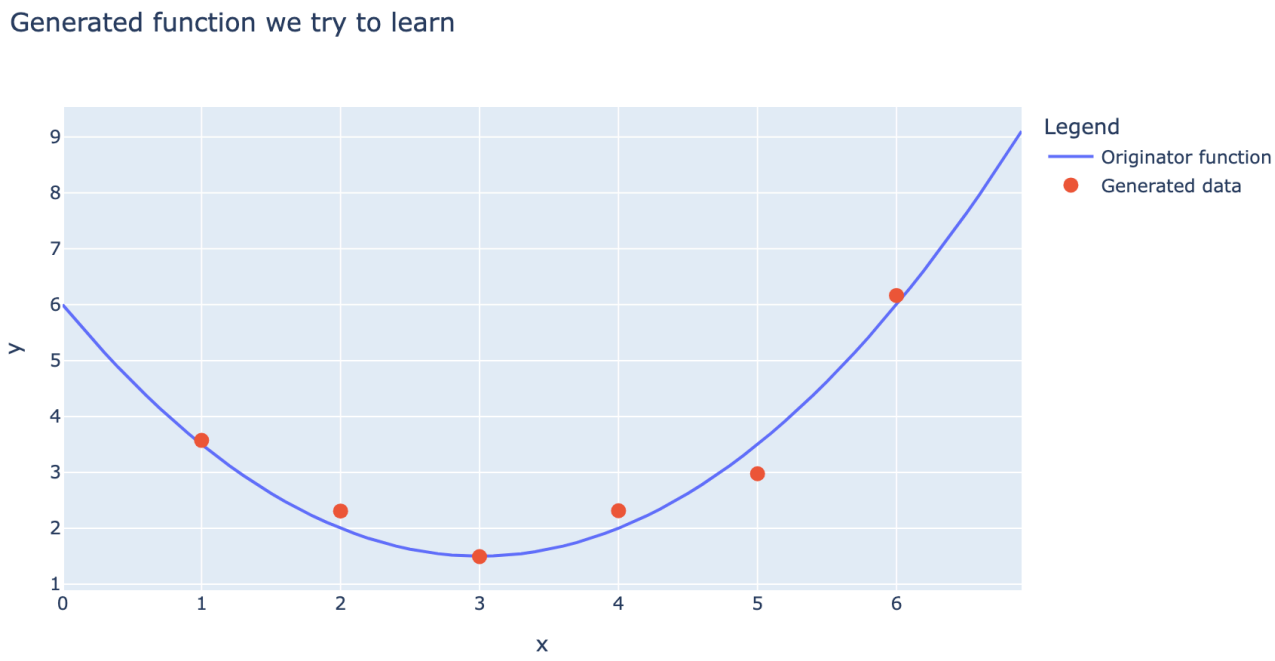
In this notebook we investigate the main idea behind Gaussian processes (GPs) and how conditional probability can be used to make predictions from data.

The idea is that there is a function that is unknown to us (the "originator function" below). We sample the function in different "experiments" and get noisy data in return. We use such data to construct a Gaussian process.

Create the originator function

Here we first create an originator function that is nonlinear and has modifiable parameters. The function is sampled at the points `XPOINTS`.

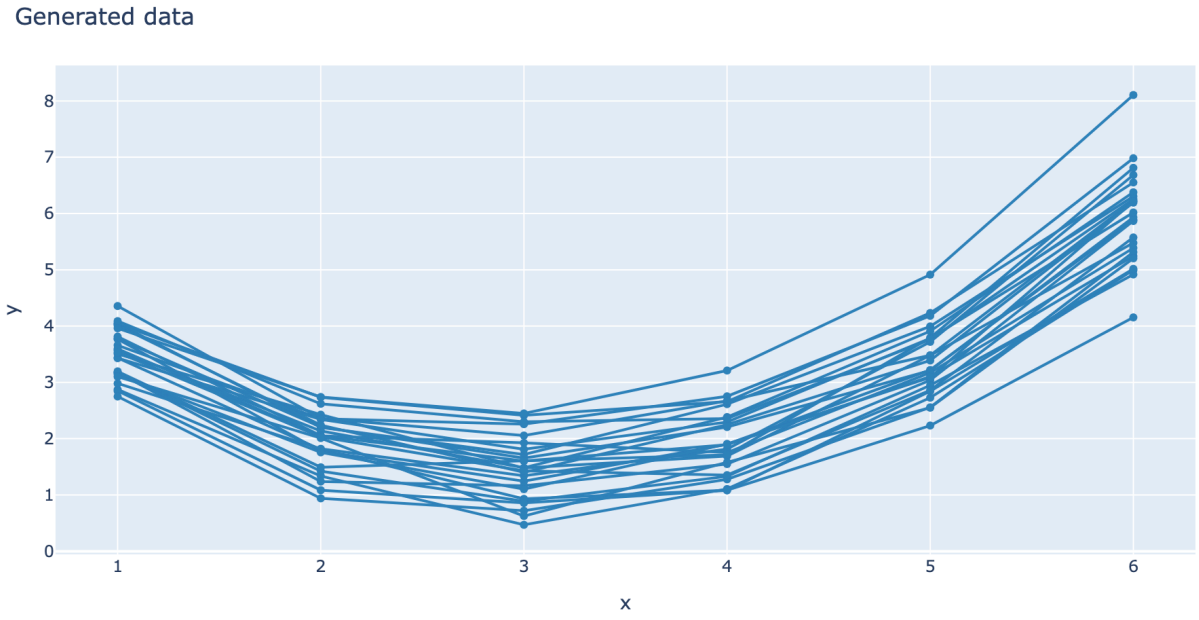
```
""" Observation points """
XPOINTS = np.array(range(1,7))
""" Function parameters """
beta0 = 6
beta1 = -3
beta2 = 0.5
sigma = 0.2
""" Function randomness """
SEED = 42
np.random.seed(SEED)
""" Originator function definition """
def f(x, beta0=6, beta1=-3, beta2=0.5, sigma=0.2):
    return np.matmul(np.vstack((np.ones(len(x)), np.array(x), np.array(x)
**2)).T,np.array([beta0,beta1,beta2])) + sigma*np.random.normal(0,1,len(x))
```



Generate data from the function

Now we generate data from that function. `NO_EXP` defines the number of experiments, i.e. how many times the underlying process is sampled at the different values of x. To generate the samples we slightly vary the underlying parameters of the function for each experiment, so every experiment is a slightly different realization of the process.

```
""" Number of experiments """
NO_EXP = 25
```

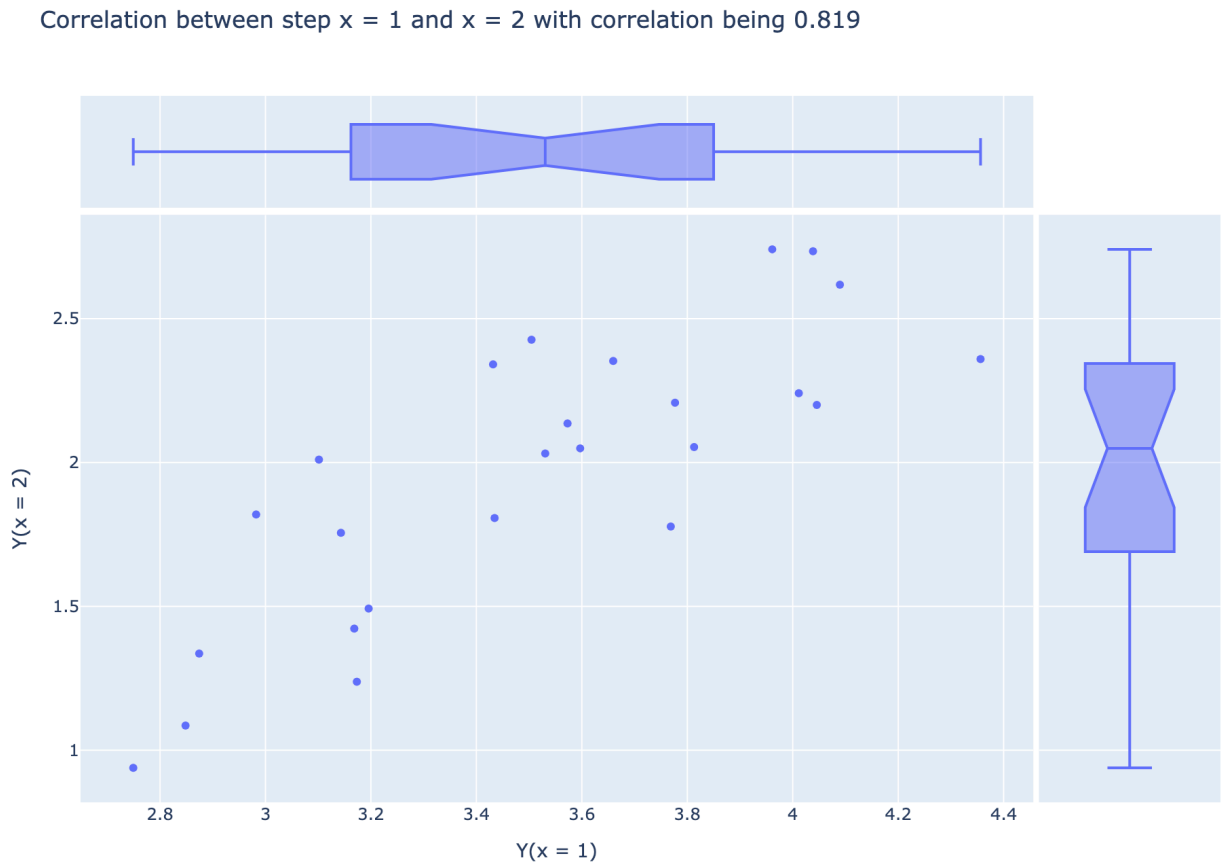


Conditional probability of the next step

Plot correlations

Here we show as an example how the data at x = 1 are correlated to those at x = 2. Below we compute the mean of the function at each sampling point and the corresponding covariance and correlation matrices.

```
""" Select steps to plot on axis """
STEP_XAXIS = 1
STEP_YAXIS = 2
```



Here we compute and visualize the mean, covariance, and correlation for all the steps.

```
Y_mu = np.mean(YEXP,axis=0)
Y_cov = np.cov(YEXP,rowvar=False)
Y_corr = np.corrcoef(YEXP,rowvar=False)
print('Mean: \n',pd.DataFrame(Y_mu,index=XPOINTS).T)
print('\nCovariance: \n',pd.DataFrame(Y_cov,index=XPOINTS,columns=XPOINTS))
print('\nCorrelation: \n',pd.DataFrame(Y_corr,index=XPOINTS,columns=XPOINTS))
```

```
Mean:
          1         2         3        4         5         6
0  3.513084  1.966933  1.477335  1.93705  3.367365  5.907705

Covariance:
          1         2         3         4         5         6
1  0.194905  0.179594  0.193249  0.198866  0.149583  0.153221
2  0.179594  0.246583  0.226285  0.232950  0.206985  0.213588
3  0.193249  0.226285  0.314380  0.278909  0.287660  0.326630
4  0.198866  0.232950  0.278909  0.342012  0.318072  0.396732
5  0.149583  0.206985  0.287660  0.318072  0.384331  0.460696
6  0.153221  0.213588  0.326630  0.396732  0.460696  0.672466

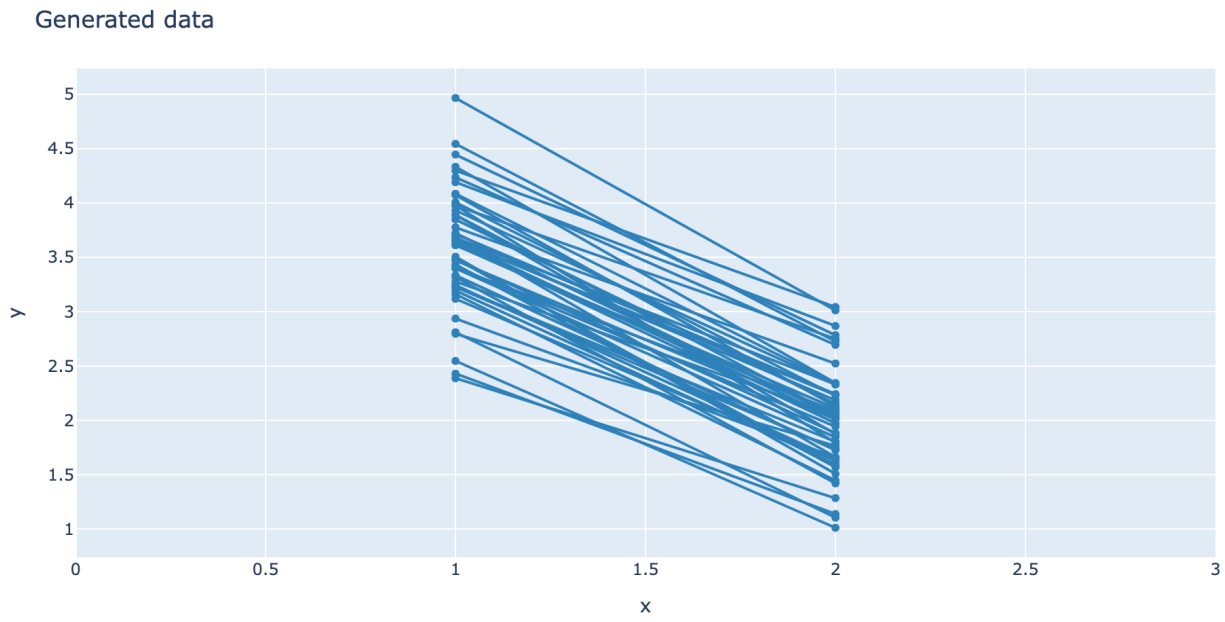
Correlation:
          1         2         3         4         5         6
1  1.000000  0.819216  0.780691  0.770245  0.546535  0.423225
2  0.819216  1.000000  0.812732  0.802160  0.672364  0.524518
3  0.780691  0.812732  1.000000  0.850579  0.827558  0.710385
4  0.770245  0.802160  0.850579  1.000000  0.877308  0.827260
5  0.546535  0.672364  0.827558  0.877308  1.000000  0.906204
6  0.423225  0.524518  0.710385  0.827260  0.906204  1.000000
```

```
# Assume a multivariate normal distribution for the data with the mean and covariance computed above
# and draw a large number of samples from it
NUM_MVN_SAMPLES = 500000
YMN = np.random.multivariate_normal(Y_mu, Y_cov, NUM_MVN_SAMPLES)
```

Using the correlations

If we define the correlation structure and suppose that the underlying probability density function is a multivariate normal distribution, we can sample from this distribution and generate new data. Below we plot pairs of sampled data at x = 1 and x = 2.

```
""" Define number of samples to generate from multivariate distribution """
NUM_SAMPLES = 50
""" Select steps to plot on axis """
STEP_XAXIS = 1
STEP_YAXIS = 2
```



Conditional probability given a single observation

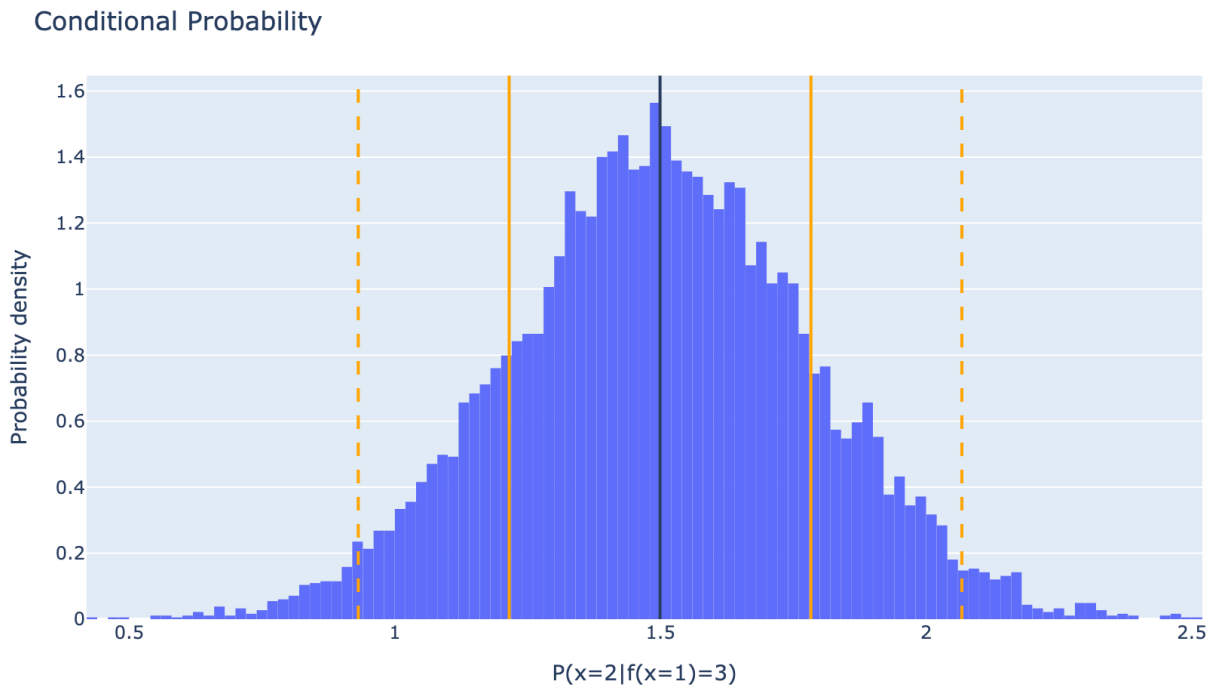
What is the distribution of $f(x)$ at $x = 2$ if we know that $f(x = 1) = 3$? This is a typical conditional probability problem, where we have to "conditionally" sample from the original distribution: we keep only the samples whose value at $x = 1$ lies within a small tolerance around the observed value.

Hence we look at the conditional probability $P(B \mid A)$ where $B : x =$ `STEP_B` and $A : f(x =$ `STEP_A` $)=$ `VALUE_A`.

```
""" Step at which we want to find out conditional probability """
STEP_B = 2
""" Step & Value at which we condition on / we have observed """
STEP_A = 1
VALUE_A= 3
""" tolerance region for observed value range """
TOL = 0.02
```

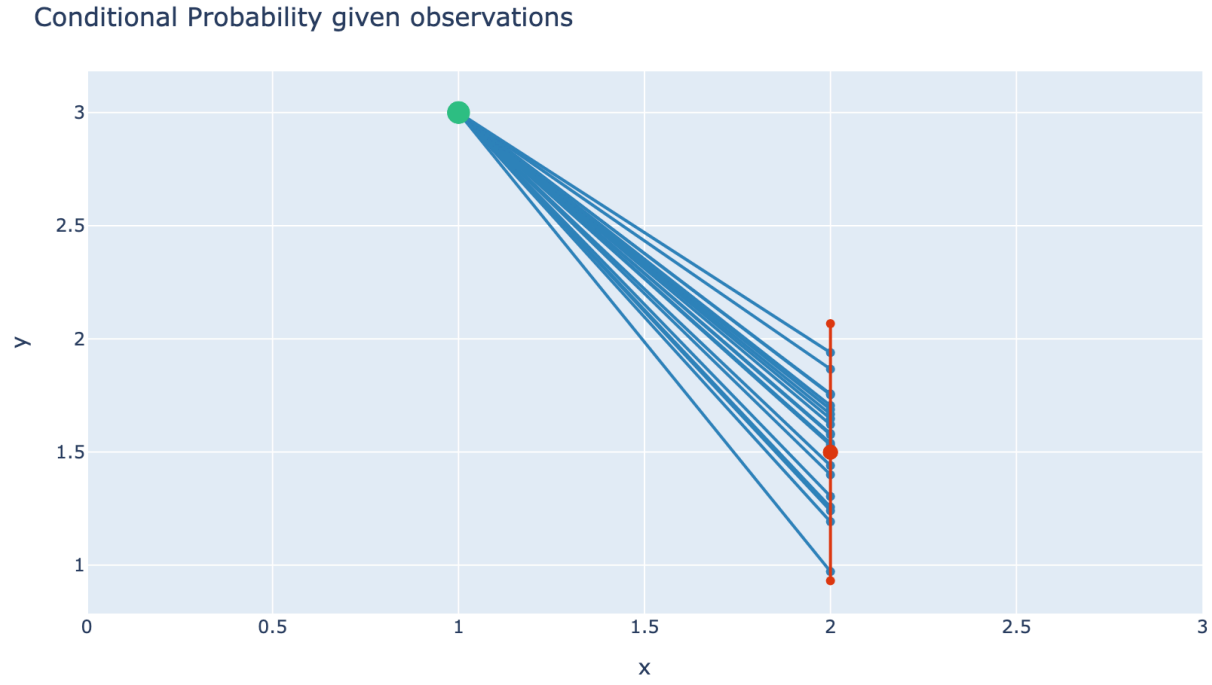
```
# Keep only the samples of the distribution that match the observation (within the
tolerance)
rows = np.logical_and(YMN[:,STEP_A-1] > VALUE_A - TOL, YMN[:,STEP_A-1] < VALUE_A +
TOL)
print('Number of matching samples: ', rows.sum())
# Compute the conditional mean and standard deviation from the matching samples
mu_step = np.mean(YMN[rows,STEP_B-1])
std_step = np.std(YMN[rows,STEP_B-1])
print('Conditional Mean: ', round(mu_step, 4))
print('Conditional Deviation: ', round(std_step, 4))
```

```
Number of matching samples:  9141
Conditional Mean:  1.4993
Conditional Deviation:  0.284
```



Hence we can show the conditional probability of the function given our observation: the lines are samples that pass through the observed point, the orange marker and bar show the conditional mean and the 2-sigma interval.

```
""" Select number of simulations to plot """
NUM_SIMULATIONS = 20
```



Conditional probability given an observation, for all steps

Histogram of f(x) at x = 2, 3, 4, 5, 6 given f(x = 1) = 3

It is possible to repeat the exercise above, where we fixed the value of $f(x=1)=3$. This time we check the probability of measuring $f(x)$ at $x=2,3,4,5,6$. As shown in the correlation matrix above, the values at these positions are correlated to those at $x=1$, although the correlation decreases further away from $x=1$, and so the corresponding uncertainty increases.

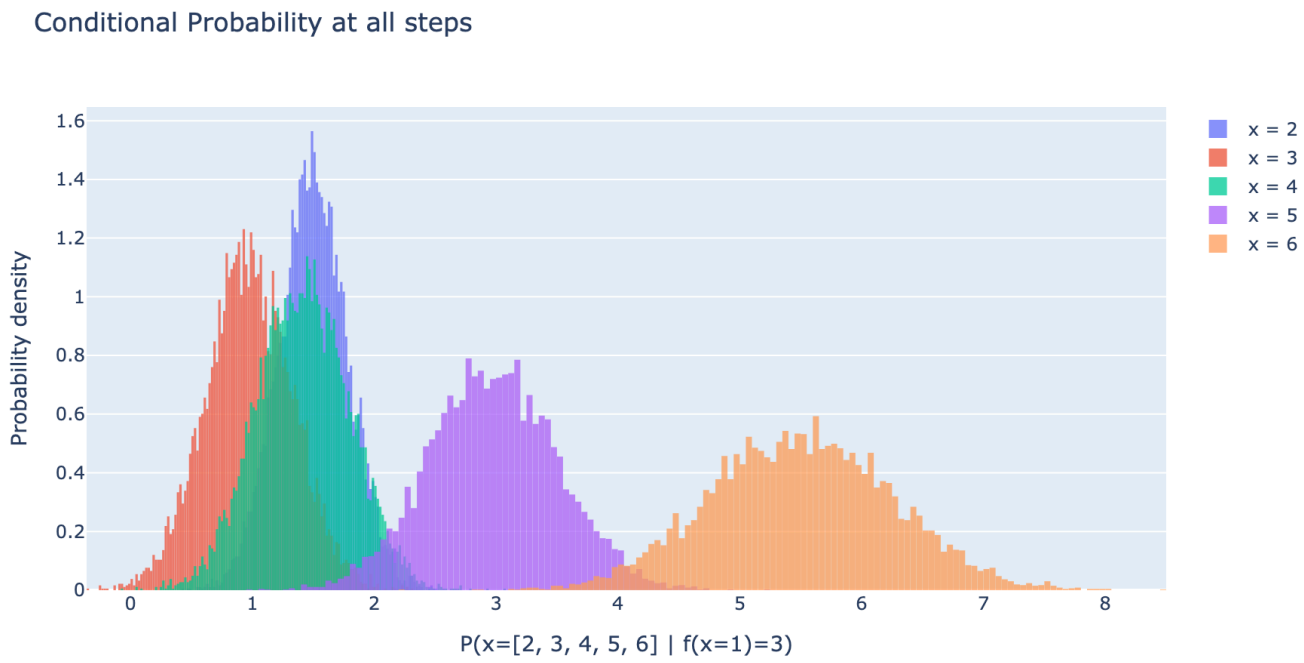
```
""" Step & Value at which we condition on / we have observed """
STEP_A = 1
VALUE_A= 3
""" tolerance region for observed value range """
TOL = 0.02
""" Compute conditional probabilities for remaining steps """
STEPS_OTHER = [2,3,4,5,6]
```

```
STEPS_ALL = copy.copy(STEPS_OTHER)
STEPS_ALL.insert(0,STEP_A)
# Keep only the samples that match the observation
rows = np.logical_and(YMN[:,STEP_A-1] > VALUE_A - TOL, YMN[:,STEP_A-1] < VALUE_A +
TOL)
print('Number of matching samples: ', rows.sum())
mu_OTHER = []
std_OTHER = []
for i in STEPS_OTHER:
    mu_OTHER.append(np.mean(YMN[rows,i-1]))
    std_OTHER.append(np.std(YMN[rows,i-1]))
df_sum = pd.DataFrame(columns = ["steps","means"])
df_sum["steps"]= STEPS_OTHER
df_sum["means"]= mu_OTHER
print('\nMeans: \n',df_sum.to_string(index=False))
Y_corr = np.corrcoef(YMN[rows][:,[i-1 for i in STEPS_OTHER]],rowvar=False)
print('\nCorrelations:
\n',pd.DataFrame(Y_corr,columns=STEPS_OTHER,index=STEPS_OTHER),'\n')
```

```
Number of matching samples:  9141

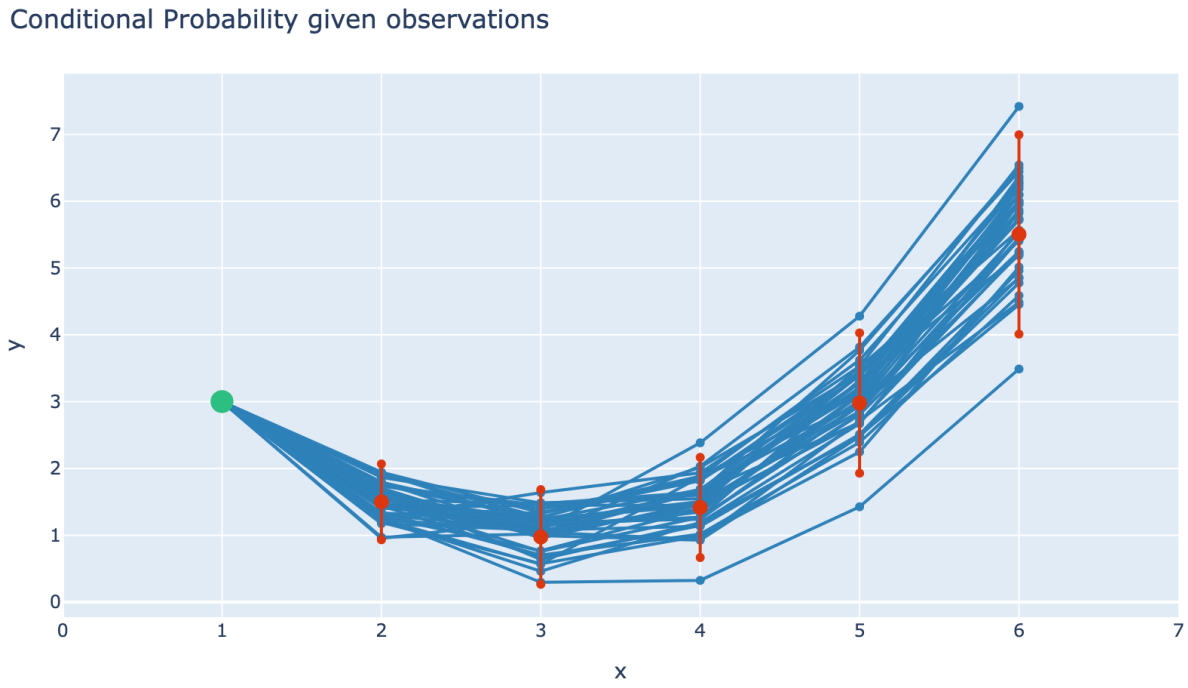
Means:
  steps    means
      2 1.499287
      3 0.973808
      4 1.415779
      5 2.978939
      6 5.504204

Correlations:
          2         3         4         5         6
2  1.000000  0.486872  0.471913  0.468463  0.347173
3  0.486872  1.000000  0.630219  0.768298  0.681114
4  0.471913  0.630219  1.000000  0.853663  0.867466
5  0.468463  0.768298  0.853663  1.000000  0.893380
6  0.347173  0.681114  0.867466  0.893380  1.000000
```



Visualizing the corresponding function realizations

```
""" Select number of simulations to plot """
NUM_SIMULATIONS = 40
```



Conditional probability given multiple observations

Let's suppose we fix the starting point at $f(x = 1) = 3$ and additionally observe $f(x = 4) = 2$. What is the corresponding distribution of the remaining values?

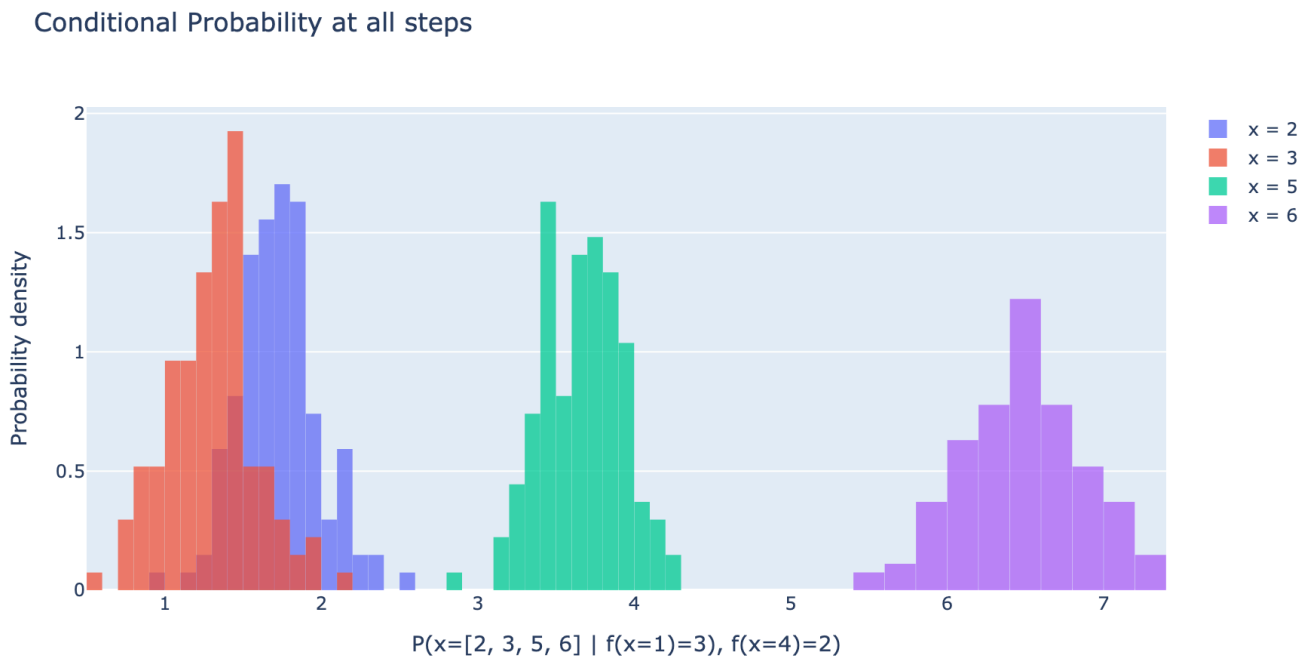
```
""" Step & Value at which we condition on / we have observed """
STEP_A = 1
VALUE_A= 3
""" Additional Step & Value at which we condition on / we have observed """
STEP_B = 4
VALUE_B= 2
""" tolerance region for observed value range """
TOL = 0.02
""" Compute conditional probabilities for remaining steps """
STEPS_OTHER = [2,3,5,6]
```

```
STEPS_ALL = copy.copy(STEPS_OTHER)
STEPS_ALL.append(STEP_A)
STEPS_ALL.append(STEP_B)
STEPS_ALL.sort()
# Keep only the samples that match both observations
rows = np.logical_and(np.logical_and(YMN[:,STEP_A-1] > VALUE_A - TOL, YMN[:,STEP_A-1]
< VALUE_A + TOL),
                      np.logical_and(YMN[:,STEP_B-1] > VALUE_B - TOL, YMN[:,STEP_B-1]
< VALUE_B + TOL))
print('Number of matching samples: ', rows.sum())
mu_OTHER = []
std_OTHER = []
for i in STEPS_OTHER:
    mu_OTHER.append(np.mean(YMN[rows,i-1]))
    std_OTHER.append(np.std(YMN[rows,i-1]))
df_sum = pd.DataFrame(columns = ["steps","means"])
df_sum["steps"]= STEPS_OTHER
df_sum["means"]= mu_OTHER
print('\nMeans: \n',df_sum.to_string(index=False))
Y_corr = np.corrcoef(YMN[rows][:,[i-1 for i in STEPS_OTHER]],rowvar=False)
print('\nCorrelations:
\n',pd.DataFrame(Y_corr,columns=STEPS_OTHER,index=STEPS_OTHER),'\n')
```

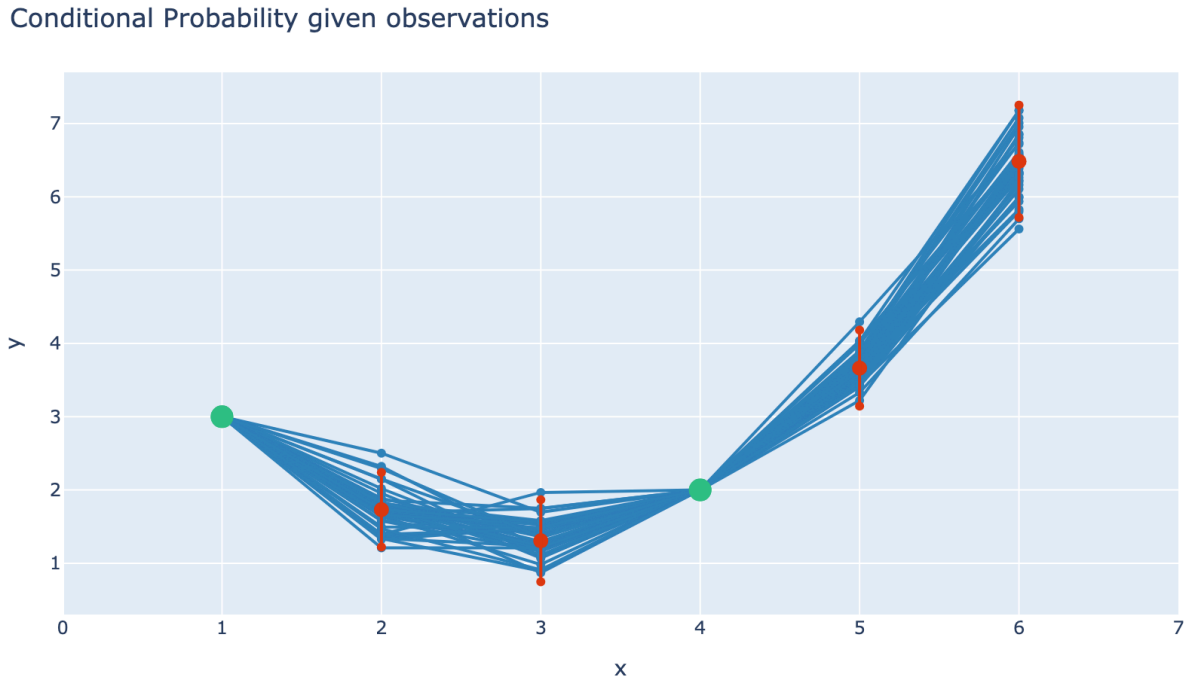
```
Number of matching samples:  135

Means:
  steps    means
      2 1.731107
      3 1.307180
      5 3.662776
      6 6.484117

Correlations:
           2         3         5         6
2  1.000000  0.186237  0.120908 -0.101213
3  0.186237  1.000000  0.620883  0.362417
5  0.120908  0.620883  1.000000  0.636384
6 -0.101213  0.362417  0.636384  1.000000
```




```
""" Select number of simulations to plot """
NUM_SIMULATIONS = 40
```



Kernels in Gaussian processes

In this section we try to learn an unknown function (the "originator function" below) from a few samples.

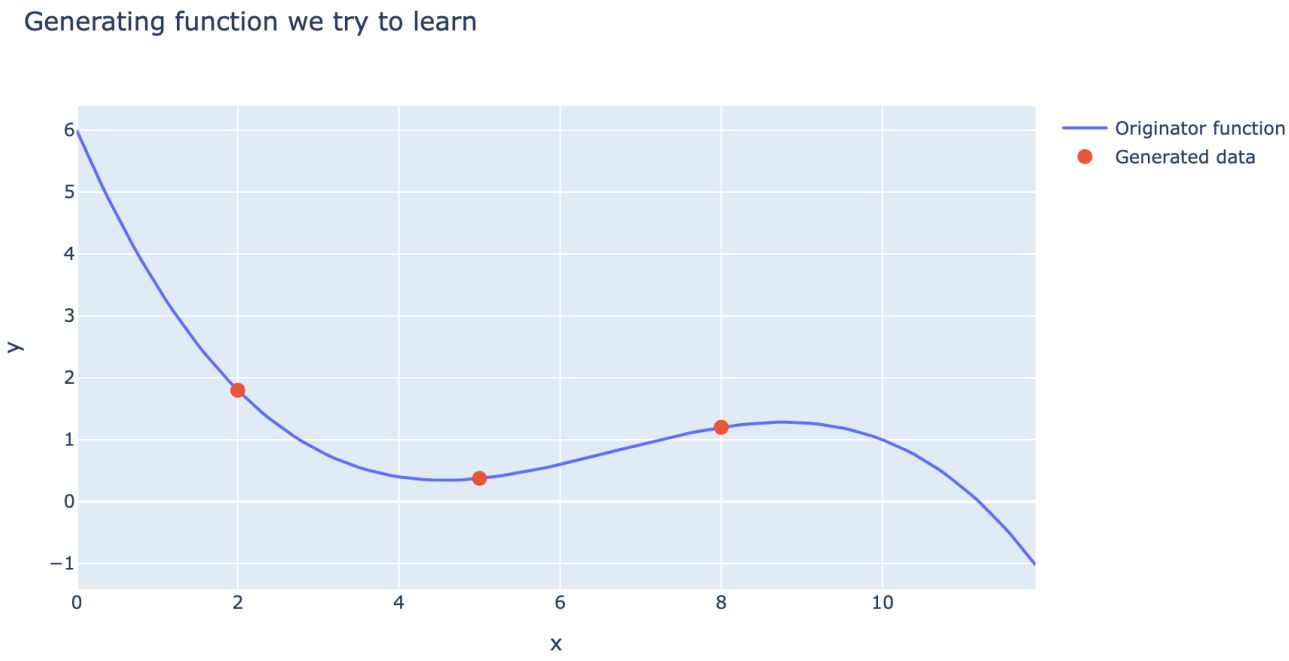
The key point is the definition of a kernel function, which defines the similarity between data points. The key parameter is `sigma`, which defines the so-called similarity scale (length scale).

In the following we show how these simple concepts can be put together to predict the value of the unknown function away from the data, and how the Gaussian process model behaves in response to changes in the model parameters.

Create the originator function

As above, we define a function which we try to learn. This time it is a cubic polynomial sampled without noise at three points.

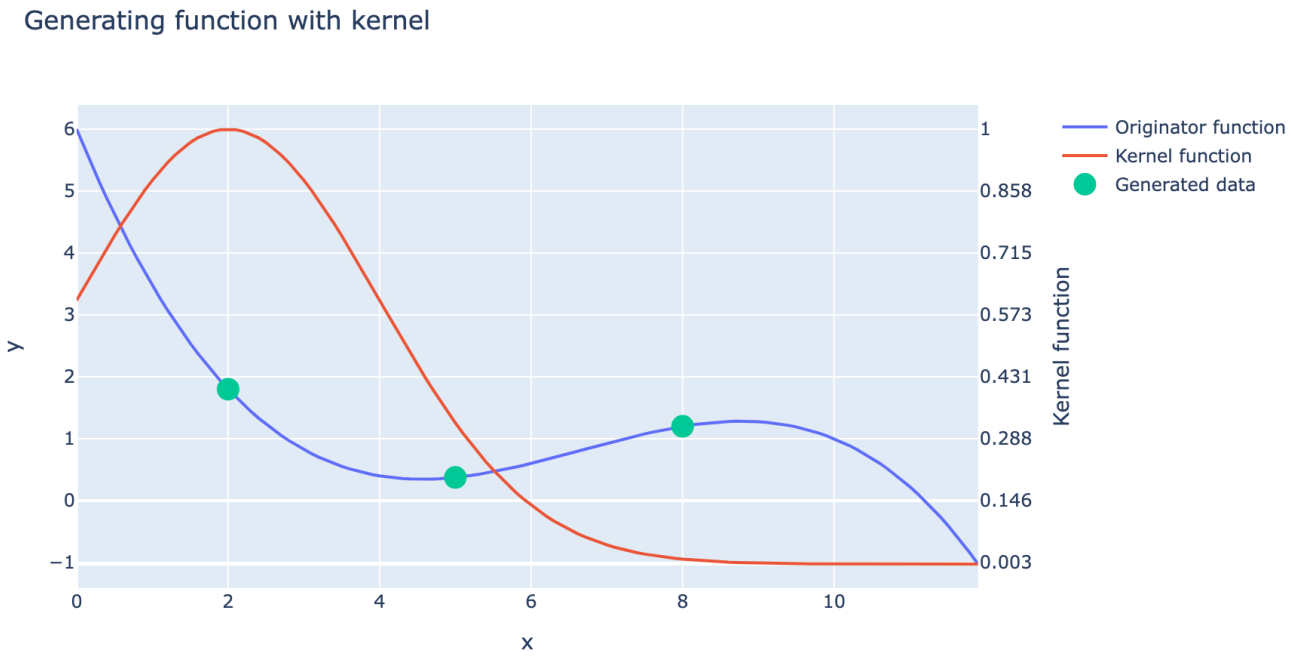
```
""" Define observations steps """
XPOINTS = np.array((2,5,8))
""" Fix seed """
np.random.seed(42)
""" Function parameters """
beta0 = 6
beta1 = -3
beta2 = 0.5
beta3 = -0.025
eps = 0
# Originator function
def f(x, beta0=6, beta1=-3, beta2=0.5, beta3=-0.025,eps=0):
    return np.matmul(np.vstack((np.ones(len(x)), np.array(x), np.array(x)
**2,np.array(x) **3)).T,np.array([beta0,beta1,beta2,beta3])) +
eps*np.random.normal(0,1,len(x))
```



Create a kernel function

You can select different values of the similarity scale `sigma`. Three kernels are prepared in the function below (uncomment the one you want to try). For better understanding, we plot the value of the similarity (kernel) function centred on the first observation (i.e. the kernel is $= 1$ at x = 2).

```
""" Select point around which we plot kernel """
PLOT_BASE = XPOINTS[0] # x=2
""" Kernel parameters """
sigma = 2
def kernel_fcn(xi,xj,sigma=2):
    ## This is rbf / normal distribution kernel
    return np.exp(-0.5*((np.array(xi)-np.array(xj))/sigma)**2)
    ## This is laplacian kernel
    # return np.exp(- sigma*np.abs((np.array(xi) - np.array(xj))))
    ## This is polynomial kernel
    # return (np.array(xi)*np.array(xj)+1)**sigma
```



Compute the kernel matrix between all pairs of observation points.

```
no_data = len(XPOINTS)
k = np.zeros((no_data,no_data))
for i in range(no_data):
    k[i,:] = kernel_fcn(XPOINTS[i],XPOINTS,sigma)
K = pd.DataFrame(k)
K.index = XPOINTS
K.columns = XPOINTS
print("Kernel given observations: \n",K)
```

```
Kernel given observations:
          2         5         8
2  1.000000  0.324652  0.011109
5  0.324652  1.000000  0.324652
8  0.011109  0.324652  1.000000
```

Add a new observation point

You can define a new point `XPOINTS_NEW` at which we want to predict the function. The new point is added in the last position of the new kernel matrix, which is now 4x4, and calculated below.

```
""" Define a new observation point """
XPOINTS_NEW = 6
```

```
# Update kernel
x_pred = np.array(XPOINTS_NEW)
x_new = np.append(XPOINTS,x_pred)
k_pred = np.zeros((len(XPOINTS)+1,len(XPOINTS)+1))
# Copy old kernel
k_pred[0:len(XPOINTS),0:len(XPOINTS)] = k
# Compute kernel for new observations
k_pred[:,len(XPOINTS)] =  kernel_fcn(x_pred,x_new,sigma)
k_pred[len(XPOINTS),:] =  kernel_fcn(x_pred,x_new,sigma)
# Visualize kernel
K_pred = pd.DataFrame(k_pred); K_pred.index = x_new ; K_pred.columns = x_new
print("\nKernel given new observations: \n",K_pred)
```

```
Kernel given new observations:
          2         5         8         6
2  1.000000  0.324652  0.011109  0.135335
5  0.324652  1.000000  0.324652  0.882497
8  0.011109  0.324652  1.000000  0.606531
6  0.135335  0.882497  0.606531  1.000000
```

Create the covariance matrix

A second parameter is needed: the prior variance of the function values `SIGMA_F2`. It scales the kernel (which is 1 on the diagonal) to the covariance matrix and expresses our prior knowledge about how much the function varies.

```
""" Prior variance of the function values """
SIGMA_F2 = 2.0
cov_pred = SIGMA_F2*k_pred
Cov_pred = pd.DataFrame(cov_pred); Cov_pred.index = x_new ; Cov_pred.columns = x_new
print("\n Covariance between points: \n",Cov_pred)
```

```
 Covariance between points:
          2         5         8         6
2  2.000000  0.649305  0.022218  0.270671
5  0.649305  2.000000  0.649305  1.764994
8  0.022218  0.649305  2.000000  1.213061
6  0.270671  1.764994  1.213061  2.000000
```

New point prediction

In Gaussian processes, the prediction at a new point x^* has the target value $f^* = f(x^*)$, which is equal to y^* in the noise-free case we are dealing with now. The case with measurement error is described below.

Given the training and test points defined above, the **joint distribution** of the Gaussian process function values over the training and test points (f and f^* respectively) is

$$\begin{bmatrix} f \\ f^* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(x,x) & K(x,x^*) \\ K(x^*,x) & K(x^*,x^*) \end{bmatrix}\right) \tag{1}$$

However, when making a new prediction, what we are actually interested in (as visualized above) is the conditional distribution of the possible predictions f^* given our train and test data and train targets x, x^*, f. This is given by:

$$f^*|x, x^*, f \sim \mathcal{N}\left(\bar{f}\left(x^*\right), Cov(\bar{f}\left(x^*\right))\right) \tag{2}$$

where the mean is:

$$\bar{f}\left(x^*\right) = K(x^*, x)K(x, x)^{-1}f \tag{3}$$

and the covariance is:

$$Cov(\bar{f}\left(x^*\right)) = K(x^*, x^*) - K(x^*, x)K(x, x)^{-1}K(x, x^*) \tag{4}$$

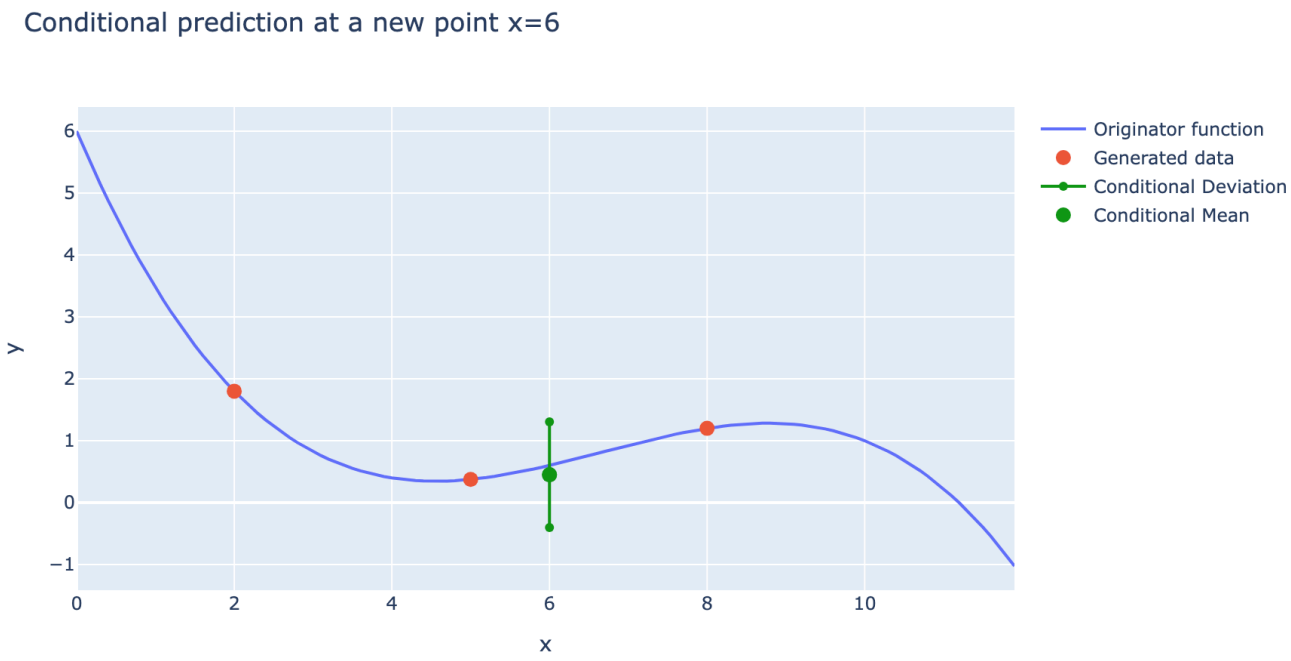
Create the conditional multivariate distribution

The function below implements the two formulas above: it computes the conditional mean and variance at the new point, given the other observations. The data are de-meaned before conditioning, because the GP prior has zero mean.

```
def mvncond2(joint_mean, joint_cov, index, obs_diff):
    # joint_mean: mean vector of the joint distribution (zero vector)
    # joint_cov:  covariance matrix including the new point
    # index:      position of the new point in the covariance matrix
    # obs_diff:   obs_f - obs_mu, the de-meaned observations
    i = index
    kxx = joint_cov[0:i,0:i]      # K(x,x)
    kxsx = joint_cov[i,0:i]       # K(x*,x)
    s12s22 = np.matmul(kxsx,np.linalg.inv(kxx))
    cond_mean = joint_mean[i] + np.matmul(s12s22,(obs_diff-joint_mean[0:i]))
    cond_cov = joint_cov[i,i] - np.matmul(s12s22,np.transpose(kxsx))
    return cond_mean, cond_cov
```

```
# Calculate the conditional mean and std at the new point
joint_mean = np.zeros(len(XPOINTS)+1)
joint_cov = cov_pred
obs_f = YPOINTS
obs_mu = np.mean(YPOINTS)
cond_mean,cond_cov = mvncond2(joint_mean,joint_cov,len(XPOINTS),obs_f - obs_mu)
new_obs_mu = cond_mean + obs_mu
print('Conditional mean of the new point: ', round(new_obs_mu,4))
new_obs_std = np.sqrt(cond_cov)
print('Conditional std of the new point:  ', round(new_obs_std,4))
```

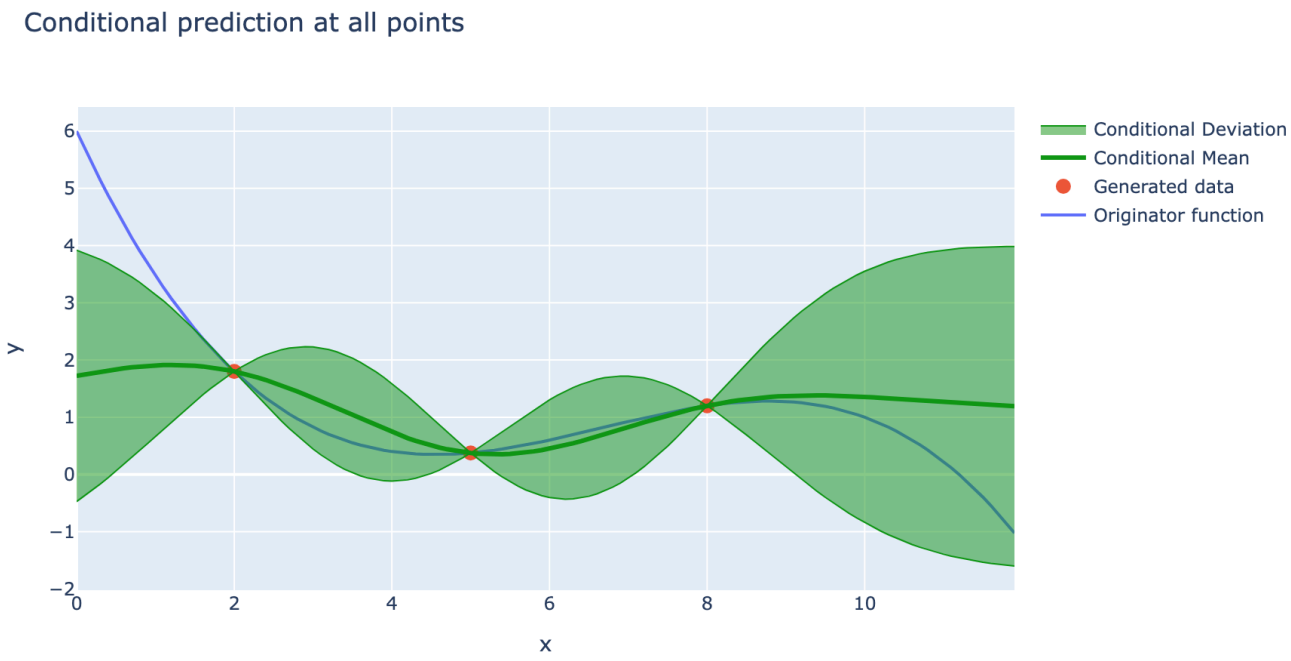
```
Conditional mean of the new point:  0.4507
Conditional std of the new point:   0.4269
```



Compute predictions for all x

Here we repeat exactly the same procedure as above for all the points of the x-axis grid. This gives the GP mean prediction and its uncertainty band.

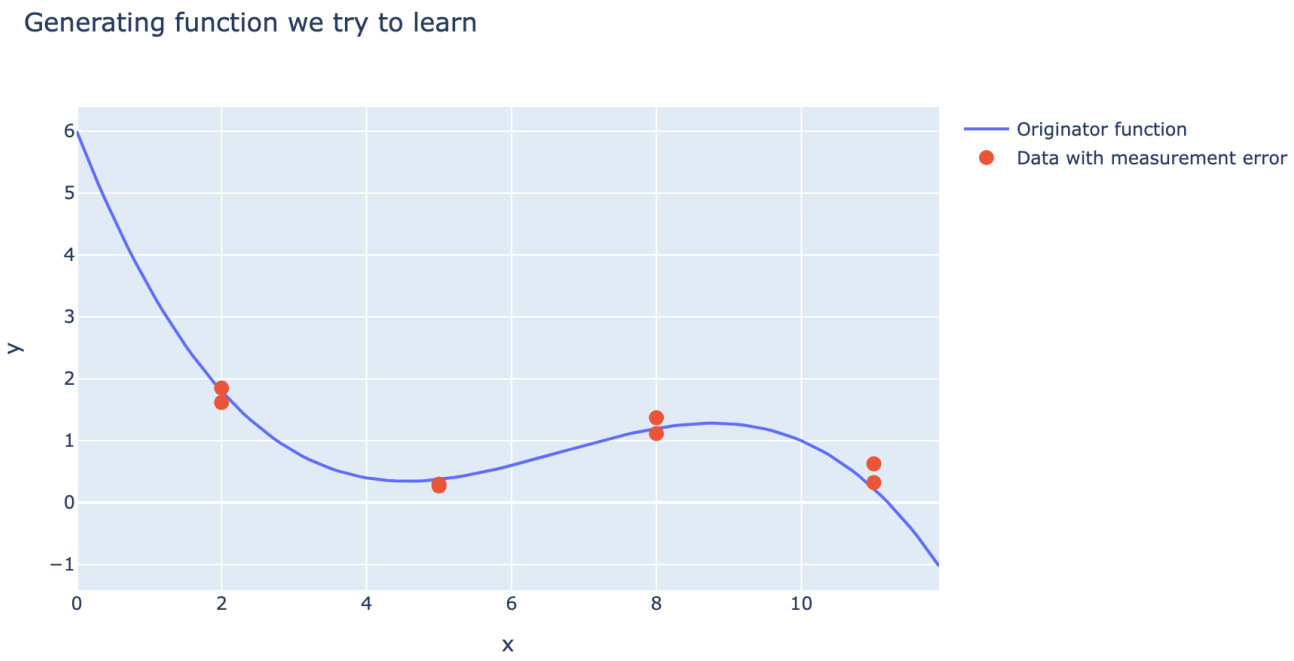
```
# Loop through all points and compute the conditional mean and std
xgrid_mu = []
xgrid_std = []
for new_obs_point in xgrid:
    # Update kernel
    x_pred = np.array(new_obs_point)
    x_new = np.append(XPOINTS,x_pred)
    k_pred = np.zeros((len(XPOINTS)+1,len(XPOINTS)+1))
    # Copy old kernel
    k_pred[0:len(XPOINTS),0:len(XPOINTS)] = k
    # Compute kernel for new observations
    k_pred[:,len(XPOINTS)] =  kernel_fcn(x_pred,x_new,sigma)
    k_pred[len(XPOINTS),:] =  kernel_fcn(x_pred,x_new,sigma)
    # Compute joint distribution
    joint_mean = np.zeros(len(XPOINTS)+1)
    joint_cov = SIGMA_F2*k_pred
    obs_f = YPOINTS
    obs_mu = np.mean(YPOINTS)
    # Compute conditional distribution
    cond_mean,cond_cov = mvncond2(joint_mean,joint_cov,len(XPOINTS),obs_f - obs_mu)
    xgrid_mu.append(cond_mean + obs_mu)
    xgrid_std.append(np.sqrt(cond_cov))
```



Effect of measurement error

In this section we simulate the effect of measurement error on the model. We sample the function twice at 4 regular points and add noise to the measurements.

```
""" Generate observations """
XPOINTS = np.arange(2,12,3)
""" Specify number of measurements at the observation points and noise """
NUM_MEASUREMENTS = 2
NOISE_MEASUREMENTS = 0.2
x = np.repeat(XPOINTS,NUM_MEASUREMENTS)
""" Fix seed """
np.random.seed(43)
```



We compute the kernel between all pairs of (noisy) observation points.

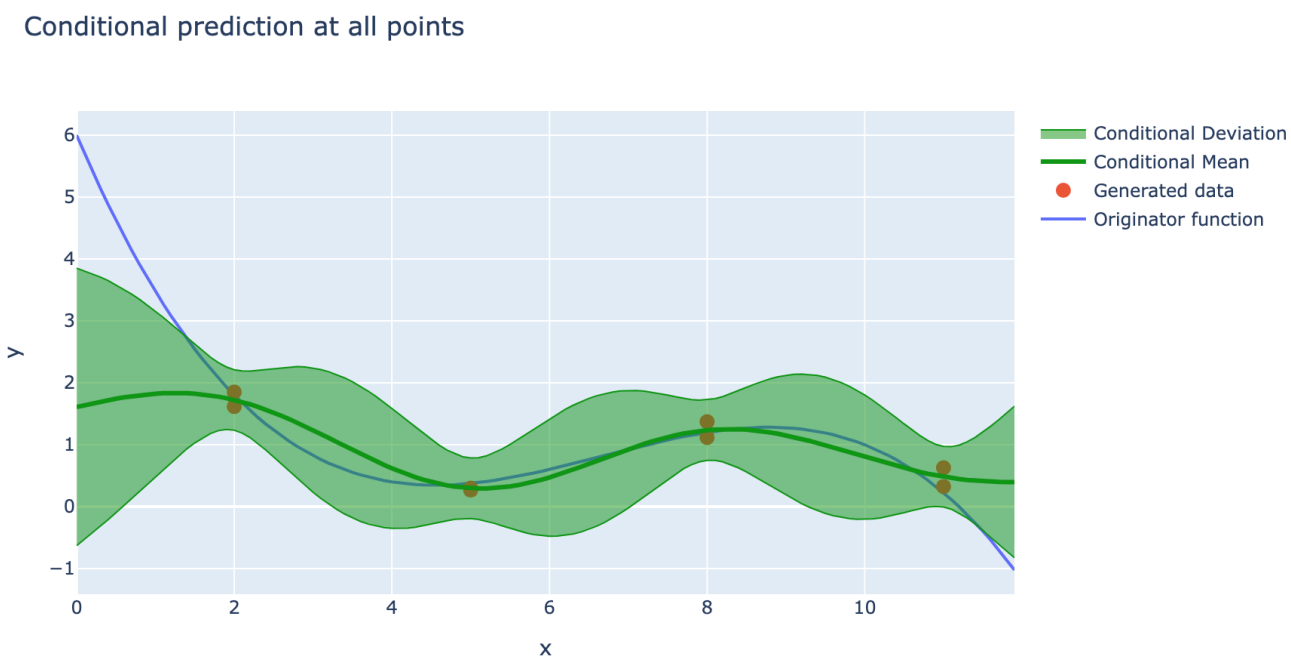
```
no_data = len(x)
k = np.zeros((no_data,no_data))
for i in range(no_data):
    k[i,:] = kernel_fcn(x[i],x,sigma)
K = pd.DataFrame(k)
K.index = x
K.columns = x
print("Kernel given observations: \n",K)
```

```
Kernel given observations:
           2         2         5         5         8         8         11  \
2   1.000000  1.000000  0.324652  0.324652  0.011109  0.011109  0.000040
2   1.000000  1.000000  0.324652  0.324652  0.011109  0.011109  0.000040
5   0.324652  0.324652  1.000000  1.000000  0.324652  0.324652  0.011109
5   0.324652  0.324652  1.000000  1.000000  0.324652  0.324652  0.011109
8   0.011109  0.011109  0.324652  0.324652  1.000000  1.000000  0.324652
8   0.011109  0.011109  0.324652  0.324652  1.000000  1.000000  0.324652
11  0.000040  0.000040  0.011109  0.011109  0.324652  0.324652  1.000000
11  0.000040  0.000040  0.011109  0.011109  0.324652  0.324652  1.000000

          11
2   0.000040
2   0.000040
5   0.011109
5   0.011109
8   0.324652
8   0.324652
11  1.000000
11  1.000000
```

And create predictions. The measurement noise variance is added to the diagonal of the covariance matrix, which tells the GP that the observations are not exact.

```
# Loop through all points and compute the conditional mean and std
xgrid_mu = []
xgrid_std = []
for new_obs_point in xgrid:
    num_data = len(x)
    # Update kernel
    x_pred = np.array(new_obs_point)
    x_new = np.append(x,x_pred)
    k_pred = np.zeros((num_data+1,num_data+1))
    # Copy old kernel
    k_pred[0:num_data,0:num_data] = k
    # Compute kernel for new observations
    k_pred[:,num_data] =  kernel_fcn(x_pred,x_new,sigma)
    k_pred[num_data,:] =  kernel_fcn(x_pred,x_new,sigma)
    # Compute joint distribution
    joint_mean = np.zeros(num_data+1)
    # IMPORTANT! we add the measurement noise variance to the diagonal of the
covariance matrix
    joint_cov = SIGMA_F2*k_pred + eps**2 * np.eye(num_data+1)
    obs_f = y
    obs_mu = np.mean(y)
    # Compute conditional distribution
    cond_mean,cond_cov = mvncond2(joint_mean,joint_cov,num_data,obs_f - obs_mu)
    xgrid_mu.append(cond_mean + obs_mu)
    xgrid_std.append(np.sqrt(cond_cov))
```



Tasks

1. Run the same setup with a different kernel (uncomment the Laplacian or the polynomial kernel in `kernel_fcn`). How does the similarity between points influence the conditional probability?
2. Change the similarity scale `sigma` to 0.5 and to 5. What happens to the prediction between and outside the observation points?

3. Increase the measurement noise `NOISE_MEASUREMENTS` to 1. How does the uncertainty band change compared to the noise-free case?

```
# Your code here
```